BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 120 · Lesson 120 · Following Algorithms and Building Trace Tables

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What drives decisions in spiral development?

Q2. Why can an array provide direct indexed access?

Q3. A full adder receives A=1, B=1 and carry-in=0. What are sum and carry-out?

Q4. For the binary value 1010, what is its decimal value?

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

trace table — Record of algorithm state as steps execute.

iteration — Repeated execution of instructions.

dry run — Manual simulation of algorithm.

boundary test — Test at edge of valid input domain.

recursion — A method calls itself with a smaller or simpler version of the problem.

base case — Stopping condition that returns without another recursive call.

New learning

Identify state — Find variables, data structures, conditions and inputs.

Trace each step — Update a table when an assignment or state change occurs. For a recursive function, record each call , its argument and its eventual return value; unlike a loop, calls wait on the call stack.

Check outcome — Compare output against constraints and edge cases. Recursion must reach a base case that returns without another recursive call, or it cannot terminate safely.

Worked example

Trace total ← 0; FOR n = 1 TO 4: IF n MOD 2 = 0 THEN total ← total + n; NEXT n; OUTPUT total.

1. Model answer — At n=1 total 0; n=2 total 2; n=3 total 2; n=4 total 6; output 6. Record n and total at each loop step. — Check: link each claim to the scenario and use precise OCR terminology.

2. Spot the stopping condition — n == 0 is the base case : return 0 immediately. Otherwise SumTo(n - 1) invokes the same method with a smaller argument. Assume n is non-negative.

3. Trace calls before returns — SumTo(3) waits for SumTo(2) , then SumTo(1) , then SumTo(0) . The first three calls remain active until the base case returns.

4. Follow the returns in reverse order — SumTo(0)=0 ; then SumTo(1)=1 , SumTo(2)=3 , SumTo(3)=6 . An iterative alternative would use a loop and running total; recursion instead retains pending calls.

We Do · Practise with guidance

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Trace largest ← 0; values=[3,8,5]; FOR each value IF value>largest THEN largest←value; OUTPUT largest.

What must we decide?

Identify the inputs, relevant facts and exact output or judgement required.

Work through the method

Use the middle stage of the visual to follow the process and state each intermediate result.

Shared worked answer

Progress: initial 0; after 3 → 3; after 8 → 8; after 5 stays 8; final output 8. The initial zero fails if all values are negative; use first element instead.

Trace factorial (product rather than sum). Predict Factorial(3) .

```
static int Factorial(int n) {
  if (n <= 1) return 1;
  return n * Factorial(n - 1);
}
// Non-negative integer input only.
```

Find the base case

Which inputs return 1 immediately? What argument does Factorial(3) pass into its next call?

Predict the order

Factorial(3) → Factorial(2) → Factorial(1) . The final call returns 1; earlier calls then resume in reverse order.

Unwind to the result

Factorial(2)=2×1=2 ; Factorial(3)=3×2=6 . A loop with result=1 and repeated multiplication could also produce 6. Neither approach automatically solves negative-input validation.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Trace count ← 0; FOR i = 1 TO 5: IF i MOD 2=1 THEN count←count+1; NEXT; OUTPUT count. [6]

Question 2

Explain why initialising max ← 0 can cause a fault when finding the maximum of negative values. [6]

Question 3

Trace recursive SumTo(4) using the method shown, name its base case and explain one difference from an iterative solution. [8]

Exit Ticket · Answer before checking

A trace table only needs the final value of each variable.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Identification, assessment and mitigation of risk.

Q2. Its element locations can be calculated from base address and index.

Q3. Sum 0 and carry-out 1.

Q4. 10.

You Do pages · Independent questions

Q1. Odd i values 1,3,5 cause increments; count progresses 1,2,3; output 3. Credit correctly labelled trace columns and output.

Q2. Zero exceeds all negative inputs [2], result incorrectly remains 0 [2], initialise from first actual element and iterate remainder [2].

Q3. Correct descending call arguments 4,3,2,1,0 [2]; base case n=0 returns 0 [2]; unwinding returns 1,3,6,10 [2]; iteration uses a loop/running total rather than pending function calls [2].

Exit Ticket · Model answer

False. It records state changes as instructions execute.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

BLACK PEN Answer all questions first.

GREEN PEN Then: tick and rewrite mistakes.

Do Now	I Do	We Do	You Do	Review & Improve
Recall previous learning	Read new knowledge	Practise with guidance	Work independently	Check understanding